

Class Project U3.

Markov Decision Processes

A mobile robot has the job of collecting empty soda cans in an office environment. It has sensors for detecting cans, and an arm and gripper that can pick them up and place them in an onboard bin; it runs on a rechargeable battery. The robot's control system has components for interpreting sensory information, for navigating, and for controlling the arm and gripper. High-level decisions about how to search for cans are made by a reinforcement learning agent based on the current charge level of the battery. To make a simple example, we assume that only two charge levels can be distinguished, comprising a small state set

$$\mathcal{S} = \{ high, low \}$$

In each state the agent can decide whether to (1) actively *search* for a can for a certain period of time, (2) remain stationary and *wait* for someone to bring it a can, or (3) head back to its home base to *recharge* its battery. When the energy level is *high*, recharging would always be foolish, so we do not include it in the action set for this state. The action sets are then

$$\mathcal{A}(high) = \{search, wait\}$$

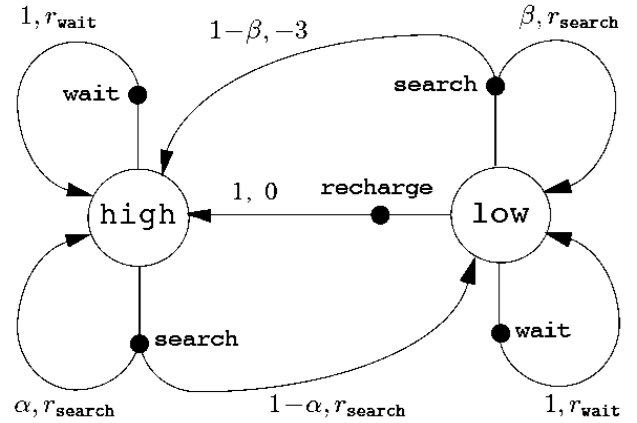
$$\mathcal{A}(low) = \{search, wait, recharge\}$$

The rewards are zero most of the time, but become positive when the robot secures an empty can, or large and negative if the battery runs all the way down. The best way to find cans is to actively search for them, but this runs down the robot's battery, whereas waiting does not. Whenever the robot is searching, the possibility exists that its battery will become depleted. In this case the robot must shut down and wait to be rescued (producing a low reward). If the energy level is *high*, then a period of active search can always be completed without risk of depleting the battery. A period of searching that begins with a *high* energy level leaves the energy level *high* with probability α and reduces it to low with probability $1 - \alpha$. On the other hand, a period of searching undertaken when the energy level is *low* leaves it *low* with probability β and depletes the battery with probability $1 - \beta$. In the latter case, the robot must be rescued, and the battery is then recharged back to *high*. Each can collected by the robot counts as a unit reward, whereas a reward of -3 results whenever the robot has to be rescued. Let r_{search} and r_{wait} , with $r_{search} > r_{wait}$, respectively denote the expected number of cans the robot will collect (and hence the expected reward) while searching and while waiting. Finally, suppose that no cans can be collected during a run home for recharging, and that no cans can be collected on a step in which the battery is depleted.



This system is then a finite MDP, and we can write down the transition probabilities and the expected rewards, with dynamics as indicated in the table on the left:

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-



Note that there is a row in the table for each possible combination of current state, s , action, $a \in \mathcal{A}(s)$, and next state, s' . Some transitions have zero probability of occurring, so no expected reward is specified for them. Shown on the right is another useful way of summarizing the dynamics of a finite MDP, as a *transition graph*. There are two kinds of nodes: *state nodes* and *action nodes*. There is a state node for each possible state (a large open circle labeled by the name of the state), and an action node for each state–action pair (a small solid circle labeled by the name of the action and connected by a line to the state node). Starting in state s and taking action a moves you along the line from state node s to action node (s, a) . Then the environment responds with a transition to the next state's node via one of the arrows leaving action node (s, a) . Each arrow corresponds to a triple (s, s', a) , where s' is the next state, and we label the arrow with the transition probability, $p(s' | s, a)$, and the expected reward for that transition, $r(s, a, s')$. Note that the transition probabilities labeling the arrows leaving an action node always sum to 1.

Task 1: Write a Python script to simulate the transition dynamics of the can-collecting robot described in the provided materials. Your implementation must follow the theoretical model, including the defined states, actions, rewards, and transition probabilities. Assign values to the parameters α and β , and use them to model the stochastic behavior of the system. Run multiple simulations and analyze the impact of different values of α and β on the robot's behavior and outcomes.

Task 2: Using the simulation developed in Task 1, generate visualizations to analyze the behavior of the system.

- Plot the frequency of each state.
- The accumulated reward over time,
- A comparison of the system's performance for different values of α and β .
- Based on your results, discuss how these parameters influence the dynamics and overall performance of the robot.

Task 3: Define and evaluate different policies for the robot. A policy specifies which action the agent takes in each state. Implement at least two different policies for comparison. Examples of policies: *always search (greedy but risky)*, *always wait (safe but non-greedy)*, or *balanced strategy (when in low, recharge)*. For each policy, run multiple simulations (at least 50 episodes per policy) and compute the average accumulated reward. Compare the results and discuss which policy performs better and why.

The project will be delivered in .ipynb format.